VOLUME I · EXAM CHEATSHEET · STUDY GUIDE

CMS 708 — Structured Programming

PGD Computer Science, Rivers State University, Nkpolu-Oroworukwo, Port Harcourt · built from the lecturer's four-module course document, with every C++ listing re-indented so it can actually be read and copied

Prepared by **Mbosinwa Awunor** · www.mbosinwa.dev

Exam: Monday 10 Aug 2026 **Time:** 11:00 – 14:00 **Venue:** the exam hall **Lecturer:** the lecturer

WHAT THIS PAPER WILL ASK YOU TO DO

There is **no past paper on file** for CMS 708, so priorities come from the shape of the course document itself. Two facts about it decide everything.

1. **Every module ends with practice exercises — 23 of them in total.** They are the only assessment-shaped material the lecturer wrote, and they are overwhelmingly of the form "*write a program that…*". Expect the paper to be dominated by **write-the-code** questions, backed by short theory parts.
2. **The theory is thin but very quotable** — structured vs unstructured, modularity and readability, by value vs by reference, scope and lifetime, top-down vs bottom-up. Each comes with a real-world analogy in the notes (traffic, wedding, house, car). **Reproduce the analogy** — it is the lecturer's own and it costs one sentence.

Where the marks are: writing correct, complete, compilable C++ · tracing code to predict output · explaining a concept then illustrating it with a short program. **All 23 exercises are solved in Volume II.**

WRITE CODE THE WAY IT IS MARKED

The C++ in the course document lost its indentation when it was converted from Word. **Do not copy that style into your answer booklet.** Every listing in this guide has been re-indented. In the exam, always: **1.** include the headers, **2.** open with `int main() {` and close with `return 0;`, **3.** indent one level per brace, **4.** end every statement with a semicolon, **5.** add a one-line comment on anything non-obvious. Marks are lost for missing headers and missing `return 0;` far more often than for wrong logic.

1 Definitions to write word-for-word

TERM	DEFINITION
Structured programming	A programming approach that emphasizes the use of **clear control structures** — sequence, selection and iteration — and **modular design** instead of arbitrary jumps, so that programs are easier to understand, test and extend. Its philosophy: **break a complex problem into smaller, manageable parts and solve each part systematically**.
Unstructured programming	An early approach, associated with languages like BASIC and assembly, relying heavily on **`goto` statements** and sequential execution. It allowed quick development but produced "**spaghetti code**".
Spaghetti code	Programs that are **tangled, difficult to read and even harder to maintain**, in which the flow of control jumps unpredictably, making debugging and modification a nightmare.
Modularity	Dividing a program into **independent units or modules**, such as functions or procedures, each performing a specific task and able to be **developed, tested and reused independently**.
Readability	Ensuring code is **understandable not only to the original author but also to others** who may maintain or extend it, through meaningful variable names, consistent indentation and clear documentation.
Variable	A **named storage location in memory that holds data**. It must be declared with a data type, which specifies the kind of values it can store.
Strongly typed	A language in which **variables must be declared before use** and **type mismatches result in errors** — C++ is strongly typed.
Operator	A **symbol that performs an operation** on variables and values. An **expression** combines variables, constants and operators to produce a result.
Function	A **block of code designed to perform a specific task**. Functions break complex problems into smaller, manageable parts and must be defined with a **return type, a name and parameters** (if any); once defined they can be **invoked (called)** from `main()` or another function.
Pass by value	A **copy** of the variable is passed; changes inside the function **do not affect** the original variable.
Pass by reference	The **actual variable** is passed, using &; changes inside the function **do affect** the original variable.
Scope	**Where in a program a variable can be accessed. Local** variables are declared inside a function and are accessible only within it; **global** variables are declared outside all functions and are accessible throughout the program.
Lifetime	**How long a variable exists in memory.** Local variables exist only while the function runs; global variables exist for the entire program execution.
Recursion	Occurs when a **function calls itself** to solve a problem. It suits problems that break down into smaller, similar sub-problems, such as factorial or Fibonacci.
Modularisation	The process of **breaking a large program into smaller, independent units called modules**, each performing a specific task and able to be developed, tested and maintained separately.

TERM	DEFINITION
Top-down design	Starts with the **big picture** and progressively breaks it into smaller details: define the overall problem, divide into sub-problems, implement each as a module. It emphasizes **stepwise refinement**.
Bottom-up design	Starts with the **smallest building blocks** and gradually integrates them into larger systems. It emphasizes **reusability**.

2 Structured vs unstructured programming CERTAIN QUESTION

BASIS	UNSTRUCTURED	STRUCTURED
Control flow	**`goto`** statements and sequential execution; the flow **jumps unpredictably**	**Clear control structures** — sequence, selection, iteration
Design	Everything in one block	**Modular** — divided into functions
Readability	Poor — "**spaghetti code**"	High — logical flow mirrors human reasoning
Debugging	A **nightmare**; errors are hard to isolate	Easy — errors are isolated within modules
Maintenance	Very difficult to modify or extend	Easy to test and extend
Scale	Workable for small tasks; **unmanageable for larger projects**	Suits large projects and team work
Associated with	Early BASIC, assembly	C, C++, Pascal

The lecturer's analogy — quote it. "*Imagine a traffic system where cars could suddenly teleport to random roads without rules — that's what unstructured programming feels like.*" And the closing line of the section: **structured programming replaces chaos with order**, ensuring programs are not only functional but maintainable in the long run.

THE SAME TASK, BOTH WAYS — THE EXAM'S FAVOURITE PAIRING

UNSTRUCTURED — using goto

```
#include <iostream>
using namespace std;

int main() {
    int x = 0;
start:
    cout << "Enter a number (0 to quit): ";
    cin >> x;
    if (x != 0) {
        cout << "You entered: " << x << endl;
        goto start;          // jumps back to start
    }
    cout << "Program ended." << endl;
    return 0;
}
```

STRUCTURED — using a do–while loop

```
#include <iostream>
using namespace std;

int main() {
    int x;
    do {
        cout << "Enter a number (0 to quit): ";
        cin >> x;
        if (x != 0) {
            cout << "You entered: " << x << endl;
        }
    } while (x != 0);
    cout << "Program ended." << endl;
    return 0;
}
```

The sentence that earns the mark: both programs do the same thing, but **the `goto` version makes the flow harder to follow, and as the program grows debugging becomes chaotic**, while **the do-while gives a clear, predictable structure that is easier to read, maintain and extend**.

Modularity and readability — the two pillars

Modularity divides a program into independent units, each performing a specific task, developed, tested and reused independently. This **reduces redundancy and enhances collaboration**, since different programmers can work on separate modules without interfering with each other.

Readability ensures code is understandable to others, through **meaningful variable names, consistent indentation and clear documentation**. It avoids unnecessary complexity and follows logical patterns that mirror human reasoning.

Together they make programming **a disciplined activity rather than a chaotic one**, transforming code into a form of communication — **not just between the programmer and the computer, but between programmers themselves**.

Why it matters in practice: teams work on large projects like banking systems, hospital management software or video games. Without modularity the codebase becomes overwhelming.

WITHOUT modularity — everything crammed into main()

```
int main() {
    int a, b;
    cout << "Enter two numbers: ";
    cin  >> a >> b;
    cout << "Sum: "        << a + b << endl;
    cout << "Difference: " << a - b << endl;
    cout << "Product: "    << a * b << endl;
    cout << "Quotient: "   << a / b << endl;
    return 0;
}   // to reuse these operations we must rewrite them
```

WITH modularity — one function per operation

```
int    add(int x, int y)      { return x + y; }
int    subtract(int x, int y) { return x - y; }
int    multiply(int x, int y) { return x * y; }
double divide(int x, int y)   { return (double)x / y; }

int main() {
    int a, b;
    cout << "Enter two numbers: ";
    cin  >> a >> b;
    cout << "Sum: "        << add(a, b)      << endl;
    cout << "Difference: " << subtract(a, b) << endl;
    cout << "Product: "    << multiply(a, b) << endl;
    cout << "Quotient: "   << divide(a, b)   << endl;
    return 0;
}
```

Note the cast in `divide`: `(double)x / y`. Without it, `10 / 3` is integer division and gives **3**, not 3.333. This is a classic exam trap.

Why C++ counts as a structured language

C++ is a **multi-paradigm** language supporting both structured and object-oriented programming, but at its core it provides all the tools needed for structured programming:

- **Control structures** — `if`, `switch`, `for`, `while` and `do-while`, which **enforce logical flow**.
- **Functions** — enabling modular design and code reuse; they can be **parameterised, recursive and organised into libraries**, making them powerful building blocks.
- **Strong typing and data abstraction** — which help programmers manage complexity and avoid errors.

The closing argument to write: although C++ is known for object-oriented features such as classes and inheritance, **its foundation lies in structured programming principles, and mastering them is essential before moving on to more advanced paradigms**.

3 C++ fundamentals

SYNTAX — THE RULES THAT ARE ALWAYS WORTH A MARK

- Every program begins with a **`main()` function**, which acts as the **entry point**.
- **Statements end with semicolons.**
- **Blocks are enclosed in curly braces `{ }`.**
- Variables **must be declared with a data type before use** — C++ is strongly typed.
- `#include` brings in a library; `using namespace std;` lets you write `cout` instead of `std::cout`.

THE SIX DATA TYPES IN THE NOTES

TYPE	HOLDS	EXAMPLE LITERAL
`int`	Integers (whole numbers)	`20`
`float`	Single-precision decimals	`3.5f`
`double`	Double-precision decimals	`45000.0`
`char`	A single character	`'A'` — single quotes
`bool`	`true` / `false`	`true`
`string`	Text — needs `<string>`	`"Ada"` — double quotes

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    int    age        = 20;        // integer
    double salary     = 45000;     // floating-point
    char   grade      = 'A';       // character
    bool   isEmployed = true;      // boolean
    string name       = "Ada";     // string

    cout << "Name: " << name
         << ", Age: " << age << endl;
    return 0;
}
```

INPUT AND OUTPUT

The `iostream` library provides **`cin` for input** and **`cout` for output**. These make programs **dynamic**, letting users supply data in real time instead of hardcoding values.

```
#include <iostream>
using namespace std;

int main() {
    int number;
    cout << "Enter a number: ";      // output prompt
    cin  >> number;                  // input from user
    cout << "You entered: " << number << endl;
    return 0;
}
```

Direction of the arrows: `cout <<` sends data **out to** the screen; `cin >>` brings data **in from** the keyboard into the variable. Writing them the wrong way round is the single most common syntax error on this paper. `endl` ends the line.

THE FIVE OPERATOR FAMILIES

FAMILY	OPERATORS
Arithmetic	`+ - * / %`
Relational	`== != < > <= >=`
Logical	`&&` (AND) · `||` (OR) · `!` (NOT)
Assignment	`= += -= *= /=`
Increment / decrement	`++ --`

> **Three operator traps. 1.** `=` assigns, `==` compares — `if (x = 5)` is always true and is a logic bug, not a syntax error. **2.** `%` is the **modulus** — the remainder — and works on integers only; `10 % 3` is **1**. It is how you test odd/even and divisibility. **3.** Integer division truncates: `10 / 3` is **3**. Cast one operand to `double` when you want 3.333.

The payroll example — everything in one program

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string name;
    double grossSalary, taxRate, netSalary;

    cout << "Enter employee name: ";
    cin  >> name;
    cout << "Enter gross salary: ";
    cin  >> grossSalary;
    cout << "Enter tax rate (%): ";
    cin  >> taxRate;

    netSalary = grossSalary - (grossSalary * taxRate / 100);

    cout << "\nEmployee: "   << name      << endl;
    cout << "Net Salary: "   << netSalary << endl;
    return 0;
}
```

This one program integrates **variables, data types, input/output and operators** — which is exactly what a "write a program that…" question is testing. Learn its shape and you can adapt it to almost any arithmetic scenario the paper sets:

1. Headers and `using namespace std;`
2. Declare every variable you will need, with the right type
3. Prompt, then read, one value at a time
4. Do the calculation in a single clearly-written statement
5. Print the result with a label
6. `return 0;`

> **Note the formula:** `net = gross − (gross × rate / 100)`. Percentage questions are common; the `/ 100` is where marks are lost.

4 Control structures

SELECTION — `IF` AND `SWITCH`

CONSTRUCT	USE IT WHEN
`if`	Executing a block **if a condition is true**
`if-else`	You need an **alternative block** when the condition is false
`switch`	**Multiple possible values of one variable** must be checked — menus, choices

`if-else` — grading system

```
int score;
cout << "Enter student score: ";
cin  >> score;

if      (score >= 70) cout << "Grade: A" << endl;
else if (score >= 60) cout << "Grade: B" << endl;
else if (score >= 50) cout << "Grade: C" << endl;
else                  cout << "Grade: F" << endl;
```

`switch` — menu selection

```
int choice;
cout << "Menu:\n1. Add\n2. Subtract\n"
     << "3. Multiply\n4. Divide\n";
cout << "Enter choice: ";
cin  >> choice;

switch (choice) {
    case 1: cout << "You chose Addition."       << endl; break;
    case 2: cout << "You chose Subtraction."    << endl; break;
    case 3: cout << "You chose Multiplication." << endl; break;
    case 4: cout << "You chose Division."       << endl; break;
    default: cout << "Invalid choice."          << endl;
}
```

Two `switch` rules that are pure marks. 1. Every case needs `break;` — without it execution **falls through** into the next case and every following case runs. **2. Always write a `default:`** for invalid input. Also: `switch` works on integers and characters, **not** on ranges or strings — a grading question must use `if-else`, not `switch`.

ITERATION — THE THREE LOOPS

LOOP	USE IT WHEN	TEST HAPPENS
`for`	The **number of iterations is known**	Before
`while`	Repeating **as long as a condition remains true**, count unknown	Before
`do-while`	Same as `while`, but it **guarantees at least one execution**	**After**

`for` — print numbers 1 to 5

```
for (int i = 1; i <= 5; i++) {
    cout << "Number: " << i << endl;
}
```

`while` — password check

```
string password;
cout << "Enter password: ";
cin  >> password;

while (password != "secret") {
    cout << "Wrong password! Try again: ";
    cin  >> password;
}
cout << "Access granted!" << endl;
```

`do-while` — repeated input

```
int num;
do {
    cout << "Enter a positive number (0 to quit): ";
    cin  >> num;
    if (num > 0) cout << "You entered: " << num << endl;
} while (num != 0);
```

The `for` header has three parts separated by semicolons: `for (initialisation; condition; update)`. Leave out the update and the loop never ends.

> **The one-line justification examiners want:** use `for` when you know how many times; `while` when you do not; `do-while` when the body **must run at least once** — which is why menus and "enter a number until 0" use it.

Nested control structures

Control structures can be **combined or nested** to handle complex logic — a loop may contain an `if`, or an `if` may contain another loop.

Nested `for` — multiplication table

```
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        cout << i << " x " << j
             << " = " << i * j << endl;
    }
    cout << endl;      // blank line between tables
}
```

Trace it: the outer loop runs 3 times; for **each** of those the inner loop runs 3 times, so the body executes **3 × 3 = 9** times. That multiplication is exactly the reasoning a "how many lines does this print?" question wants.

Operators inside a condition — voting eligibility

```
#include <iostream>
using namespace std;

int main() {
    int  age;
    bool hasID;

    cout << "Enter age: ";
    cin  >> age;
    cout << "Do you have an ID? (1 for yes, 0 for no): ";
    cin  >> hasID;

    if (age >= 18 && hasID) {
        cout << "You are eligible to vote." << endl;
    } else {
        cout << "You are not eligible to vote." << endl;
    }
    return 0;
}
```

Relational operators produce a `bool`, so they can be combined with `&&`, `||` and `!`. Note that `hasID` is already a `bool`, so `&& hasID` needs no `== true`.

5 Functions and modular programming

ANATOMY OF A FUNCTION

```
double area(double length, double width) {
//  ↑        ↑     ↑
//  return   name  parameter list
    return length * width;
}

int main() {
    double l = 5.0, w = 3.0;
    cout << "Area: " << area(l, w) << endl;   // invocation
    return 0;
}
```

A function must be defined with a **return type, a name and parameters** (if any); once defined it can be **invoked from `main()` or another function**. Here `area()` is **defined once and reused**, which is modularity in one line.

Use `void` as the return type when the function returns nothing, and then write no `return` value.

SCOPE AND LIFETIME

```
#include <iostream>
using namespace std;

int globalVar = 100;              // global variable

void showLocal() {
    int localVar = 50;            // local variable
    cout << "Local variable: "  << localVar  << endl;
    cout << "Global variable: " << globalVar << endl;
}

int main() {
    showLocal();
    cout << "Global in main: " << globalVar << endl;
    // cout << localVar;   // ERROR: not accessible here
    return 0;
}
```

Scope = where a variable can be accessed. **Lifetime** = how long it exists in memory. Local variables exist **only while the function runs**; global variables exist **for the entire program execution**. The commented-out line is the exam's favourite illustration — say **why** it fails.

PARAMETER PASSING — THE GUARANTEED COMPARISON

```
BY VALUE — a copy is passed
void incrementByValue(int x) {
    x++;
    cout << "Inside function: " << x << endl;
}

int main() {
    int num = 10;
    incrementByValue(num);
    cout << "Outside function: " << num << endl;
    return 0;
}

OUTPUT:   Inside function: 11
          Outside function: 10   ← original unchanged
```

```
BY REFERENCE — the actual variable is passed
void incrementByReference(int &x) {     // note the &
    x++;
    cout << "Inside function: " << x << endl;
}

int main() {
    int num = 10;
    incrementByReference(num);
    cout << "Outside function: " << num << endl;
    return 0;
}

OUTPUT:   Inside function: 11
          Outside function: 11   ← original modified
```

BASIS	BY VALUE	BY REFERENCE
What is passed	A **copy**	The **actual variable**
Syntax	`int x`	`int &x`
Effect on original	**None**	**Modified**
Memory	Extra copy made	No copy
Use when	The function only needs to **read** the value	The function must **change** the caller's variable, e.g. a swap

Recursion

```
#include <iostream>
using namespace std;

int factorial(int n) {
    if (n == 0) return 1;                 // base case
    else        return n * factorial(n - 1);  // recursive call
}

int main() {
    int num = 5;
    cout << "Factorial of " << num
         << " is " << factorial(num) << endl;
    return 0;
}
```

Recursion occurs when a function **calls itself**. It suits problems that break into **smaller, similar sub-problems** — factorial, Fibonacci. **Every recursion needs a base case** that stops it; the notes warn that recursion **must be used carefully to avoid infinite loops or excessive memory usage**.

TRACE IT — `FACTORIAL(5)`

```
factorial(5) = 5 * factorial(4)
factorial(4) = 4 * factorial(3)
factorial(3) = 3 * factorial(2)
factorial(2) = 2 * factorial(1)
factorial(1) = 1 * factorial(0)
factorial(0) = 1                ← base case, unwinding starts

  = 1 * 1  = 1
  = 2 * 1  = 2
  = 3 * 2  = 6
  = 4 * 6  = 24
  = 5 * 24 = 120
```

Write the trace out like this in the exam — a "show how the recursion evaluates" question is asking for exactly these two columns: the winding down to the base case and the unwinding back up.

6 Modularisation and program design LIKELY THEORY QUESTION

Modularisation is the process of breaking a large program into smaller, independent units called **modules**, each performing a specific task and able to be developed, tested and maintained separately. **This mirrors how we solve problems in everyday life: instead of tackling everything at once, we divide tasks into manageable parts.**

The lecturer's wedding analogy — write it. *"Imagine planning a wedding. Instead of one person handling everything, responsibilities are divided into modules: catering, decoration, music, photography and guest management. Each team focuses on its module, and when combined, the wedding runs smoothly."*

In programming, modularisation ensures that: code is **reusable** (a function written once can be used in many places) · programs are **easier to debug** (errors are isolated within modules) · teams can **collaborate** (different programmers work on different modules).

TOP-DOWN DESIGN

Starts with the **big picture** and progressively breaks it down. Define the overall problem, divide it into sub-problems, implement each as a module. It emphasizes **stepwise refinement**.

Analogy: building a house — the architect first designs the overall structure (rooms, floors, layout), then each part is broken down into plumbing, wiring, roofing and painting. **Overall design first, then the details.**

```
// High-level function
void manageStudent() {
    cout << "Managing student records..." << endl;
    // Refined later into smaller tasks
    cout << "1. Add student\n"
         << "2. Delete student\n"
         << "3. View student\n";
}

int main() {
    manageStudent();     // start with the big picture
    return 0;
}
```

The broad task "manage student records" is later refined into `addStudent()`, `deleteStudent()` and `viewStudent()`.

BOTTOM-UP DESIGN

Takes the opposite approach: start with the **smallest building blocks** and gradually integrate them into larger systems. Implement reusable components first, then combine them. It emphasizes **reusability**.

Analogy: assembling a car — engineers first design the engine, tyres and seats; once these modules are ready they are integrated into the complete car. **Details first, then the overall system.**

```
// Small building blocks
double add(double a, double b)      { return a + b; }
double multiply(double a, double b) { return a * b; }

// Larger system built from those modules
void calculate() {
    double x = 4, y = 2;
    cout << "Sum: "     << add(x, y)      << endl;
    cout << "Product: " << multiply(x, y) << endl;
}

int main() {
    calculate();     // integrates smaller modules
    return 0;
}
```

ASPECT	TOP-DOWN DESIGN	BOTTOM-UP DESIGN
Starting point	Big picture — the overall problem	Small modules — the building blocks
Focus	**Stepwise refinement**	**Reusability** of components
Real-life example	An architect designing a house	An engineer assembling car parts
Programming approach	Define main tasks, then refine	Build small functions, then integrate

The concluding sentence the notes give — do not omit it: both approaches are valuable, and in practice programmers often use a **hybrid approach**, combining top-down planning with bottom-up implementation to balance clarity and reusability.

7 Things you can lose easy marks on

1. Omitting `#include <iostream>` or `using namespace std;`. Write them every single time, and `<string>` whenever you use `string`.
2. Forgetting `return 0;` at the end of `main()`.
3. Writing `cin <<` or `cout >>`. Output goes **out** with `<<`, input comes **in** with `>>`.
4. Using `=` where `==` is meant inside an `if`.
5. Missing `break;` in a `switch` — every case falls through to the next.
6. Integer division: `10 / 3` gives **3**. Cast to `double` when a decimal is wanted.
7. A loop with no update statement, so it never terminates.
8. A recursive function with **no base case** — infinite recursion and a crash.
9. Forgetting the `&` when the question says "pass by reference" — the function then silently changes nothing.
10. Using a local variable outside its function and expecting it to compile.
11. Confusing `'A'` (char, single quotes) with `"A"` (string, double quotes).
12. Writing flat, unindented code. **Readability is an examinable topic on this course** — losing marks for unreadable code in a structured-programming exam is avoidable.

TIMING PLAN FOR A 3-HOUR PAPER

Read everything first and **start with the coding question you can already picture** — code either comes out or it does not, and starting with a solid one banks marks while you are fresh. Budget roughly **30 minutes per question**. For any "write a program" part, spend the first two minutes writing the **skeleton** — headers, `main()`, variable declarations — because that structure scores even if the logic in the middle is imperfect. Then fill in the logic, then **trace it once with a sample input** before moving on: a two-minute trace catches the missing `break;` or the off-by-one that would otherwise cost the whole question. Leave the pure theory parts for last — they are the fastest to write under time pressure.